

```
In [1]: 1 import pandas as pd
        2 cars=pd.read_csv('cars_data.csv')
        3 cars
```

Out[1]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mi
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	
...	...	...	...	...	...	...	...	...
6014	Maruti Swift VDI	Delhi	2014	27365	Diesel	Manual	First	
6015	Hyundai Xcent 1.1 CRDi S	Jaipur	2015	100000	Diesel	Manual	First	
6016	Mahindra Xylo D4 BSIV	Jaipur	2012	55000	Diesel	Manual	Second	
6017	Maruti Wagon R VXI	Kolkata	2013	46000	Petrol	Manual	First	
6018	Chevrolet Beat Diesel	Hyderabad	2011	47000	Diesel	Manual	First	

6019 rows × 13 columns



Information about the cars dataframe

In [2]: 1 cars.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  6019 non-null   str
1   Location              6019 non-null   str
2   Year                  6019 non-null   int64
3   Kilometers_Driven     6019 non-null   int64
4   Fuel_Type             6019 non-null   str
5   Transmission          6019 non-null   str
6   Owner_Type            6019 non-null   str
7   Mileage               6017 non-null   str
8   Engine                5983 non-null   str
9   Power                 5983 non-null   str
10  Seats                 5977 non-null   float64
11  New_Price             824 non-null    str
12  Price                 6019 non-null   float64
dtypes: float64(2), int64(2), str(9)
memory usage: 611.4 KB
```

Descriptive statistics

In [3]: 1 cars.describe()

Out[3]:

	Year	Kilometers_Driven	Seats	Price
count	6019.000000	6.019000e+03	5977.000000	6019.000000
mean	2013.358199	5.873838e+04	5.278735	9.479468
std	3.269742	9.126884e+04	0.808840	11.187917
min	1998.000000	1.710000e+02	0.000000	0.440000
25%	2011.000000	3.400000e+04	5.000000	3.500000
50%	2014.000000	5.300000e+04	5.000000	5.640000
75%	2016.000000	7.300000e+04	5.000000	9.950000
max	2019.000000	6.500000e+06	10.000000	160.000000

Display the unique values in the column location

In [4]: 1 cars['Location'].unique()

```
<StringArray>
[    'Mumbai',      'Pune',    'Chennai', 'Coimbatore', 'Hyderabad',
     'Jaipur',      'Kochi',    'Kolkata',    'Delhi', 'Bangalore',
     'Ahmedabad']
Length: 11, dtype: str
```

Frequency of each location

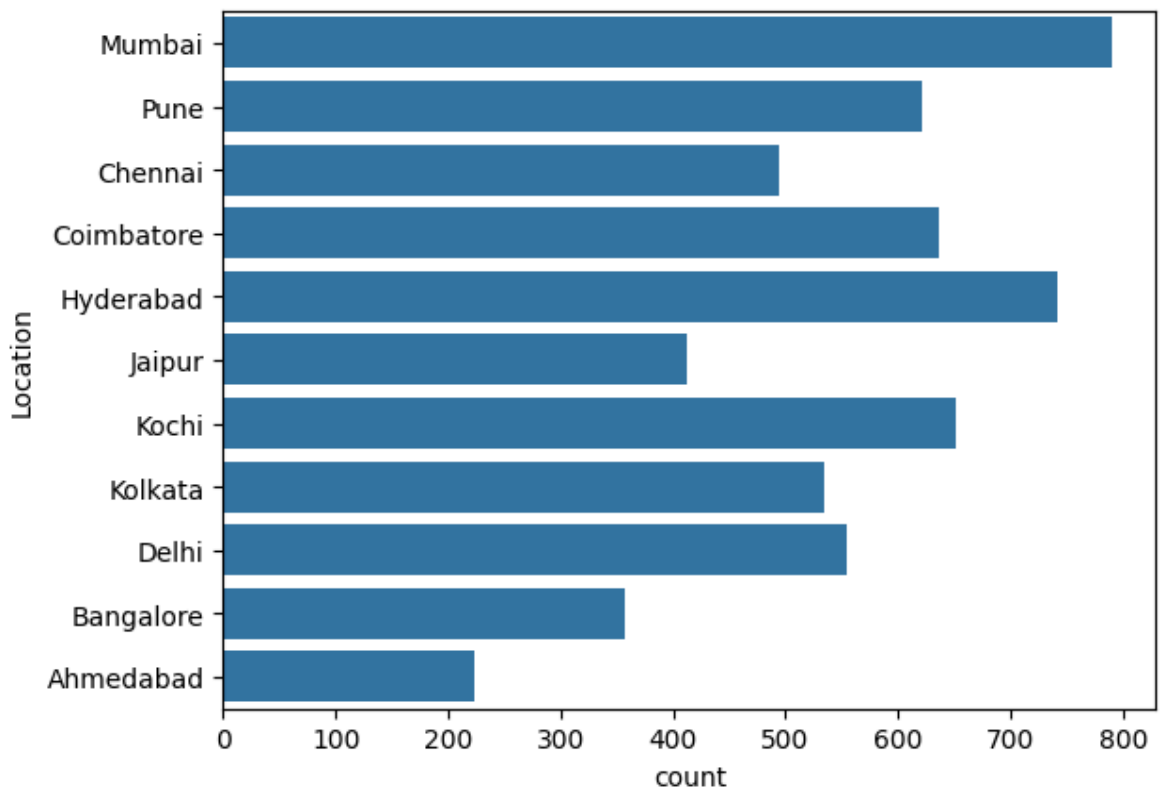
```
In [5]: 1 cars['Location'].value_counts()
```

```
Out[5]: Location
Mumbai      790
Hyderabad   742
Kochi       651
Coimbatore  636
Pune        622
Delhi       554
Kolkata     535
Chennai     494
Jaipur      413
Bangalore   358
Ahmedabad   224
Name: count, dtype: int64
```

visualize the above data usind count plot

```
In [6]: 1 import seaborn as sns
        2 sns.countplot(cars,y='Location')
```

```
Out[6]: <Axes: xlabel='count', ylabel='Location'>
```



Find the average price of the cars in each location

```
In [7]: 1 cars.groupby('Location')['Price'].mean()
```

```
Out[7]: Location
Ahmedabad      8.457321
Bangalore      13.328631
Chennai         7.753623
Coimbatore     15.078223
Delhi           9.707726
Hyderabad       9.776550
Jaipur          5.809225
Kochi           11.179186
Kolkata         5.661215
Mumbai          9.429329
Pune            6.726254
Name: Price, dtype: float64
```

Pivot table - mean Price by Location and Fuel_Type

```
In [8]: 1 cars.pivot_table(values='Price',
2                        index='Location',
3                        columns='Fuel_Type',
4                        aggfunc='mean')
```

```
Out[8]:
```

	Fuel_Type	CNG	Diesel	Electric	LPG	Petrol
Location						
Ahmedabad		3.006667	10.859420	NaN	NaN	4.660482
Bangalore		NaN	17.245388	NaN	NaN	7.157626
Chennai		4.100000	10.413080	13.00	NaN	3.958960
Coimbatore		NaN	20.747275	NaN	NaN	7.870429
Delhi		3.010000	12.554510	NaN	NaN	6.328992
Hyderabad		2.316667	11.955731	NaN	2.775	5.202643
Jaipur		NaN	7.964304	NaN	2.560	3.106630
Kochi		4.293750	15.515888	NaN	NaN	7.452714
Kolkata		NaN	6.773594	NaN	NaN	4.640538
Mumbai		4.118235	14.288626	12.75	1.450	6.315764
Pune		2.946923	9.851076	NaN	1.650	4.083312

```
In [9]: 1 cars.pivot_table(values='Price',
2               index='Location',
3               columns='Fuel_Type',
4               aggfunc='count')
```

Out[9]:

	Fuel_Type	CNG	Diesel	Electric	LPG	Petrol
Location						
Ahmedabad		3.0	138.0	NaN	NaN	83.0
Bangalore		NaN	219.0	NaN	NaN	139.0
Chennai		2.0	289.0	1.0	NaN	202.0
Coimbatore		NaN	356.0	NaN	NaN	280.0
Delhi		10.0	306.0	NaN	NaN	238.0
Hyderabad		3.0	506.0	NaN	6.0	227.0
Jaipur		NaN	230.0	NaN	2.0	181.0
Kochi		8.0	304.0	NaN	NaN	339.0
Kolkata		NaN	256.0	NaN	NaN	279.0
Mumbai		17.0	313.0	1.0	1.0	458.0
Pune		13.0	288.0	NaN	1.0	320.0

The same count using the function value_counts()

```
In [10]: 1 cars[['Location', 'Fuel_Type']].value_counts()
```

```
Out[10]: Location    Fuel_Type
Hyderabad    Diesel      506
Mumbai        Petrol      458
Coimbatore    Diesel      356
Kochi          Petrol      339
Pune           Petrol      320
Mumbai        Diesel      313
Delhi          Diesel      306
Kochi          Diesel      304
Chennai        Diesel      289
Pune           Diesel      288
Coimbatore     Petrol      280
Kolkata        Petrol      279
               Diesel      256
Delhi          Petrol      238
Jaipur         Diesel      230
Hyderabad      Petrol      227
Bangalore      Diesel      219
Chennai        Petrol      202
Jaipur         Petrol      181
```

```
In [11]: 1 cars[['Location', 'Fuel_Type']].value_counts(sort=False)
```

```
Out[11]: Location    Fuel_Type    count
Mumbai      CNG         17
Pune        Diesel      288
Chennai     Petrol      202
            Diesel      289
Coimbatore  Diesel      356
Hyderabad   LPG         6
Jaipur      Diesel      230
Mumbai      Diesel      313
Kochi       Petrol      339
Kolkata     Petrol      279
Delhi       Diesel      306
Kochi       Diesel      304
Bangalore   Diesel      219
Mumbai      Petrol      458
Hyderabad   Diesel      506
            Petrol      227
Delhi       Petrol      238
Kolkata     Diesel      256
Coimbatore  Petrol      222
```

```
In [12]: 1 cars[['Location', 'Fuel_Type']].value_counts(sort=False)
```

```
Out[12]: Location    Fuel_Type    count
Mumbai      CNG         17
Pune        Diesel      288
Chennai     Petrol      202
            Diesel      289
Coimbatore  Diesel      356
Hyderabad   LPG         6
Jaipur      Diesel      230
Mumbai      Diesel      313
Kochi       Petrol      339
Kolkata     Petrol      279
Delhi       Diesel      306
Kochi       Diesel      304
Bangalore   Diesel      219
Mumbai      Petrol      458
Hyderabad   Diesel      506
            Petrol      227
Delhi       Petrol      238
Kolkata     Diesel      256
Coimbatore  Petrol      280
Jaipur      Petrol      181
Bangalore   Petrol      139
Ahmedabad   Diesel      138
Pune        Petrol      320
Ahmedabad   Petrol       83
Pune        CNG         13
Kochi       CNG         8
Delhi       CNG         10
Hyderabad   CNG         3
Mumbai      LPG         1
Ahmedabad   CNG         3
Chennai     CNG         2
Pune        LPG         1
Jaipur      LPG         2
Chennai     Electric     1
Mumbai      Electric     1
Name: count, dtype: int64
```

Data Cleaning

1. Remove Duplicates
2. Handling Null Values
3. Handling Outliers
4. Handle Invalid values
5. Remove the irrelevant rows or columns

Find the duplicate records

```
In [13]: 1 cars.duplicated()
```

```
Out[13]: 0      False
          1      False
          2      False
          3      False
          4      False
          ...
        6014    False
        6015    False
        6016    False
        6017    False
        6018    False
        Length: 6019, dtype: bool
```

```
In [14]: 1 cars.duplicated().sum()
```

```
Out[14]: np.int64(0)
```

2. Handling null values

```
In [15]: 1 cars.isnull().sum()
```

```
Out[15]: Name      0
          Location  0
          Year      0
          Kilometers_Driven  0
          Fuel_Type  0
          Transmission  0
          Owner_Type  0
          Mileage    2
          Engine     36
          Power      36
          Seats      42
          New_Price  5195
          Price      0
          dtype: int64
```

Remove the column 'New_Price' since it has more null values

In [16]: `1 cars.drop('New_Price',axis=1)`

Out[16]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mi
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	
...	...	...	...	...	...	...	...	...
6014	Maruti Swift VDI	Delhi	2014	27365	Diesel	Manual	First	
6015	Hyundai Xcent 1.1 CRDi S	Jaipur	2015	100000	Diesel	Manual	First	
6016	Mahindra Xylo D4 BSIV	Jaipur	2012	55000	Diesel	Manual	Second	
6017	Maruti Wagon R VXI	Kolkata	2013	46000	Petrol	Manual	First	
6018	Chevrolet Beat Diesel	Hyderabad	2011	47000	Diesel	Manual	First	

6019 rows × 12 columns



In [17]: `1 cars=cars.drop('New_Price',axis=1)
2 cars.isnull().sum()`

Out[17]:

Name	0
Location	0
Year	0
Kilometers_Driven	0
Fuel_Type	0
Transmission	0
Owner_Type	0
Mileage	2
Engine	36
Power	36
Seats	42
Price	0

dtype: int64

```
In [18]: 1 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Name                   6019 non-null  str    
1   Location               6019 non-null  str    
2   Year                   6019 non-null  int64  
3   Kilometers_Driven     6019 non-null  int64  
4   Fuel_Type              6019 non-null  str    
5   Transmission           6019 non-null  str    
6   Owner_Type             6019 non-null  str    
7   Mileage                6017 non-null  str    
8   Engine                 5983 non-null  str    
9   Power                  5983 non-null  str    
10  Seats                  5977 non-null  float64
11  Price                  6019 non-null  float64
dtypes: float64(2), int64(2), str(8)
memory usage: 564.4 KB
```

the columns Mileage, Engine and Power are 'str', since with has the unit suffixed with the values

In [19]:

1 cars

Out[19]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mi
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	
...	...	...	...	...	...	...	...	...
6014	Maruti Swift VDI	Delhi	2014	27365	Diesel	Manual	First	
6015	Hyundai Xcent 1.1 CRDi S	Jaipur	2015	100000	Diesel	Manual	First	
6016	Mahindra Xylo D4 BSIV	Jaipur	2012	55000	Diesel	Manual	Second	
6017	Maruti Wagon R VXI	Kolkata	2013	46000	Petrol	Manual	First	
6018	Chevrolet Beat Diesel	Hyderabad	2011	47000	Diesel	Manual	First	

6019 rows × 12 columns



In [20]:

```
1 S="26.6 km/kg"
2 print(S.split())
```

['26.6', 'km/kg']

In [21]:

```
1 S="26.6 km/kg"
2 print(S.split()[0])
```

26.6

In [22]:

```
1 S="26.6 km/kg"
2 print(S.split()[0],type(S.split()[0]))
```

26.6 <class 'str'>

```
In [23]: 1 S="26.6 km/kg"
          2 print(float(S.split()[0]),type(float(S.split()[0])))
```

```
26.6 <class 'float'>
```

1. Split the values by space
2. Extract the 0th index value
3. Convert to numeric

```
In [24]: 1 cars['Mileage'].str.split()
```

```
Out[24]: 0      [26.6, km/kg]
          1      [19.67, kmp1]
          2      [18.2, kmp1]
          3      [20.77, kmp1]
          4      [15.2, kmp1]
          ...
        6014     [28.4, kmp1]
        6015     [24.4, kmp1]
        6016     [14.0, kmp1]
        6017     [18.9, kmp1]
        6018     [25.44, kmp1]
        Name: Mileage, Length: 6019, dtype: object
```

```
In [25]: 1 cars['Mileage'].str.split().str[0]
```

```
Out[25]: 0      26.6
          1      19.67
          2      18.2
          3      20.77
          4      15.2
          ...
        6014     28.4
        6015     24.4
        6016     14.0
        6017     18.9
        6018     25.44
        Name: Mileage, Length: 6019, dtype: object
```

Split the numerical part from Mileage

```
In [26]: 1 cars['Mileage']=cars['Mileage'].str.split().str[0]
          2 cars
```

Out[26]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mi
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	
...	...	...	...	...	...	...	...	...
6014	Maruti Swift VDI	Delhi	2014	27365	Diesel	Manual	First	
6015	Hyundai Xcent 1.1 CRDi S	Jaipur	2015	100000	Diesel	Manual	First	
6016	Mahindra Xylo D4 BSIV	Jaipur	2012	55000	Diesel	Manual	Second	
6017	Maruti Wagon R VXI	Kolkata	2013	46000	Petrol	Manual	First	
6018	Chevrolet Beat Diesel	Hyderabad	2011	47000	Diesel	Manual	First	

6019 rows × 12 columns



In [27]:

```
1 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  6019 non-null  str
1   Location              6019 non-null  str
2   Year                  6019 non-null  int64
3   Kilometers_Driven    6019 non-null  int64
4   Fuel_Type            6019 non-null  str
5   Transmission         6019 non-null  str
6   Owner_Type          6019 non-null  str
7   Mileage              6017 non-null  object
8   Engine               5983 non-null  str
9   Power               5983 non-null  str
10  Seats               5977 non-null  float64
11  Price               6019 non-null  float64
dtypes: float64(2), int64(2), object(1), str(7)
memory usage: 564.4+ KB
```

Convert the Mileage to Numeric (float)

In [28]:

```
1 cars['Mileage']=pd.to_numeric(cars['Mileage'])
2 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  6019 non-null  str
1   Location              6019 non-null  str
2   Year                  6019 non-null  int64
3   Kilometers_Driven    6019 non-null  int64
4   Fuel_Type            6019 non-null  str
5   Transmission         6019 non-null  str
6   Owner_Type          6019 non-null  str
7   Mileage              6017 non-null  float64
8   Engine               5983 non-null  str
9   Power               5983 non-null  str
10  Seats               5977 non-null  float64
11  Price               6019 non-null  float64
dtypes: float64(3), int64(2), str(7)
memory usage: 564.4 KB
```

Extract numerical part and convert the Engine to numeric

```
In [29]: 1 cars['Engine']=cars['Engine'].str.split().str[0]
          2 cars['Engine']=pd.to_numeric(cars['Engine'])
          3 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  6019 non-null  str
1   Location              6019 non-null  str
2   Year                  6019 non-null  int64
3   Kilometers_Driven    6019 non-null  int64
4   Fuel_Type            6019 non-null  str
5   Transmission         6019 non-null  str
6   Owner_Type          6019 non-null  str
7   Mileage              6017 non-null  float64
8   Engine               5983 non-null  float64
9   Power               5983 non-null  str
10  Seats               5977 non-null  float64
11  Price              6019 non-null  float64
dtypes: float64(4), int64(2), str(6)
memory usage: 564.4 KB
```

There are some invalid values in Power, First Replace that with 0

In [30]: 1 cars[cars['Power']=='null bhp']

Out[30]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mile
76	Ford Fiesta 1.4 SXi TDCi	Jaipur	2008	111111	Diesel	Manual	First	1
79	Hyundai Santro Xing XL	Hyderabad	2005	87591	Petrol	Manual	First	
89	Hyundai Santro Xing XO	Hyderabad	2007	73745	Petrol	Manual	First	1
120	Hyundai Santro Xing XL eRLX Euro III	Mumbai	2005	102000	Petrol	Manual	Second	1
143	Hyundai Santro Xing XO eRLX Euro II	Kochi	2008	80759	Petrol	Manual	Third	1
...	...	...	...	...	...	...	...	...
5873	Hyundai Santro Xing XO eRLX Euro II	Pune	2006	47200	Petrol	Manual	Second	1
5893	Maruti Estilo LXI	Chennai	2008	51000	Petrol	Manual	Second	1
5925	Skoda Laura Classic 1.8 TSI	Pune	2010	85000	Petrol	Manual	First	1
5943	Mahindra Jeep MM 540 DP	Chennai	2002	75000	Diesel	Manual	First	
5985	Toyota Etios GD	Delhi	2013	70000	Diesel	Manual	First	2

107 rows × 12 columns



In [31]: 1 cars['Power']=cars['Power'].replace('null bhp','0 bhp')

Extract numerical part and convert the Power to numeric


```
In [32]: 1 cars['Power']=cars['Power'].str.split().str[0]
          2 cars['Power']=pd.to_numeric(cars['Power'])
          3 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  6019 non-null  str
1   Location              6019 non-null  str
2   Year                  6019 non-null  int64
3   Kilometers_Driven     6019 non-null  int64
4   Fuel_Type             6019 non-null  str
5   Transmission          6019 non-null  str
6   Owner_Type            6019 non-null  str
7   Mileage               6017 non-null  float64
8   Engine                5983 non-null  float64
9   Power                 5983 non-null  float64
10  Seats                 5977 non-null  float64
11  Price                 6019 non-null  float64
dtypes: float64(5), int64(2), str(5)
memory usage: 564.4 KB
```

Fill the null values of Mileage,Engine,Power with mean value

```
In [33]: 1 cars['Mileage']=cars['Mileage'].fillna(cars['Mileage'].mean())
          2 cars['Engine']=cars['Engine'].fillna(cars['Engine'].mean())
          3 cars['Power']=cars['Power'].fillna(cars['Power'].mean())
          4 cars['Seats']=cars['Seats'].fillna(cars['Seats'].mean())
```

```
In [34]: 1 cars.isnull().sum()
```

```
Out[34]: Name                0
Location              0
Year                  0
Kilometers_Driven    0
Fuel_Type             0
Transmission          0
Owner_Type            0
Mileage               0
Engine                0
Power                 0
Seats                 0
Price                 0
dtype: int64
```

Check the numerical columns with zeros

```
In [35]: 1 cars.select_dtypes(['int64','float64'])
```

```
Out[35]:
```

	Year	Kilometers_Driven	Mileage	Engine	Power	Seats	Price
0	2010	72000	26.60	998.0	58.16	5.0	1.75
1	2015	41000	19.67	1582.0	126.20	5.0	12.50
2	2011	46000	18.20	1199.0	88.70	5.0	4.50
3	2012	87000	20.77	1248.0	88.76	7.0	6.00
4	2013	40670	15.20	1968.0	140.80	5.0	17.74
...	...	...	...	...	...	...	...
6014	2014	27365	28.40	1248.0	74.00	5.0	4.75
6015	2015	100000	24.40	1120.0	71.00	5.0	4.00
6016	2012	55000	14.00	2498.0	112.00	8.0	2.90
6017	2013	46000	18.90	998.0	67.10	5.0	2.65
6018	2011	47000	25.44	936.0	57.60	5.0	2.50

6019 rows × 7 columns

```
In [36]: 1 cars.select_dtypes(['int64','float64']).columns
```

```
Out[36]: Index(['Year', 'Kilometers_Driven', 'Mileage', 'Engine', 'Power', 'Seats',
               'Price'],
              dtype='str')
```

```
In [37]: 1 cols=cars.select_dtypes(['int64','float64']).columns
         2 for col in cols:
         3     print(col,len(cars[cars[col]==0]))
```

```
Year 0
Kilometers_Driven 0
Mileage 68
Engine 0
Power 107
Seats 1
Price 0
```

Replace the zeros in Mileage, Power and Seats with mean

```
In [38]: 1 cars['Mileage']=cars['Mileage'].replace(0,cars['Mileage'].mean())
         2 cars['Power']=cars['Power'].replace(0,cars['Power'].mean())
         3 cars['Seats']=cars['Seats'].replace(0,cars['Seats'].mean())
```

```
In [39]: 1 cars.info()
```

```
<class 'pandas.DataFrame'>  
RangeIndex: 6019 entries, 0 to 6018  
Data columns (total 12 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   Name                  6019 non-null  str      
1   Location              6019 non-null  str      
2   Year                  6019 non-null  int64    
3   Kilometers_Driven     6019 non-null  int64    
4   Fuel_Type             6019 non-null  str      
5   Transmission          6019 non-null  str      
6   Owner_Type            6019 non-null  str      
7   Mileage               6019 non-null  float64  
8   Engine                6019 non-null  float64  
9   Power                 6019 non-null  float64  
10  Seats                 6019 non-null  float64  
11  Price                 6019 non-null  float64  
dtypes: float64(5), int64(2), str(5)  
memory usage: 564.4 KB
```

In [40]:

1 cars

Out[40]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mi
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	
...	...	...	...	...	...	...	...	...
6014	Maruti Swift VDI	Delhi	2014	27365	Diesel	Manual	First	
6015	Hyundai Xcent 1.1 CRDi S	Jaipur	2015	100000	Diesel	Manual	First	
6016	Mahindra Xylo D4 BSIV	Jaipur	2012	55000	Diesel	Manual	Second	
6017	Maruti Wagon R VXI	Kolkata	2013	46000	Petrol	Manual	First	
6018	Chevrolet Beat Diesel	Hyderabad	2011	47000	Diesel	Manual	First	

6019 rows × 12 columns

***Derive two columns Brand and Model from the Name column***

```
In [41]: 1 cars['Brand']=cars['Name'].str.split().str[0]
          2 cars['Model']=cars['Name'].str.split().str[1]
          3 cars
```

Out[41]:

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mi
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	
...	...	...	...	...	...	...	...	...
6014	Maruti Swift VDI	Delhi	2014	27365	Diesel	Manual	First	
6015	Hyundai Xcent 1.1 CRDi S	Jaipur	2015	100000	Diesel	Manual	First	
6016	Mahindra Xylo D4 BSIV	Jaipur	2012	55000	Diesel	Manual	Second	
6017	Maruti Wagon R VXI	Kolkata	2013	46000	Petrol	Manual	First	
6018	Chevrolet Beat Diesel	Hyderabad	2011	47000	Diesel	Manual	First	

6019 rows × 14 columns



Remove the Name Column

```
In [42]: 1 cars=cars.drop('Name',axis=1)
```

```
In [43]: 1 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Location              6019 non-null  str    
1   Year                  6019 non-null  int64  
2   Kilometers_Driven    6019 non-null  int64  
3   Fuel_Type             6019 non-null  str    
4   Transmission          6019 non-null  str    
5   Owner_Type            6019 non-null  str    
6   Mileage               6019 non-null  float64
7   Engine                6019 non-null  float64
8   Power                 6019 non-null  float64
9   Seats                 6019 non-null  float64
10  Price                 6019 non-null  float64
11  Brand                 6019 non-null  object 
12  Model                 6019 non-null  object 
dtypes: float64(5), int64(2), object(2), str(4)
memory usage: 611.4+ KB
```

There are some non-numerical Categorical Columns Location, Fuel_Type, Transmission, Owner_Type, Brand, Model

```
In [44]: 1 cars['Fuel_Type'].value_counts()
```

```
Out[44]: Fuel_Type
Diesel      3205
Petrol      2746
CNG         56
LPG         10
Electric     2
Name: count, dtype: int64
```

LabelEncoder - It is technique to encode the non-numerical categorical data to integers

```
In [45]: 1 from sklearn.preprocessing import LabelEncoder
2 le_location=LabelEncoder() #Create an object
3 #Understand the data and transform that to integer
4 cars['Location']=le_location.fit_transform(cars['Location'])
5 cars
```

Out[45]:

	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine
0	9	2010	72000	CNG	Manual	First	26.60	998.0
1	10	2015	41000	Diesel	Manual	First	19.67	1582.0
2	2	2011	46000	Petrol	Manual	First	18.20	1199.0
3	2	2012	87000	Diesel	Manual	First	20.77	1248.0
4	3	2013	40670	Diesel	Automatic	Second	15.20	1968.0
...	...	...	...	...	...	...	...	...
6014	4	2014	27365	Diesel	Manual	First	28.40	1248.0
6015	6	2015	100000	Diesel	Manual	First	24.40	1120.0
6016	6	2012	55000	Diesel	Manual	Second	14.00	2498.0
6017	8	2013	46000	Petrol	Manual	First	18.90	998.0
6018	5	2011	47000	Diesel	Manual	First	25.44	936.0

6019 rows × 13 columns



```
In [46]: 1 le_location.classes_
```

```
Out[46]: array(['Ahmedabad', 'Bangalore', 'Chennai', 'Coimbatore', 'Delhi',
                'Hyderabad', 'Jaipur', 'Kochi', 'Kolkata', 'Mumbai', 'Pune'],
              dtype=object)
```

```
In [47]: 1 from sklearn.preprocessing import LabelEncoder
2 le_ft=LabelEncoder() #Create an object
3 #Understand the data and transform that to integer
4 cars['Fuel_Type']=le_ft.fit_transform(cars['Fuel_Type'])
```

```
In [48]: 1 from sklearn.preprocessing import LabelEncoder
2 le_wt=LabelEncoder() #Create an object
3 #Understand the data and transform that to integer
4 cars['Owner_Type']=le_wt.fit_transform(cars['Owner_Type'])
```

```
In [49]: 1 from sklearn.preprocessing import LabelEncoder
2 le_trans=LabelEncoder() #Create an object
3 #Understand the data and transform that to integer
4 cars['Transmission']=le_trans.fit_transform(cars['Transmission'])
```

```
In [50]: 1 from sklearn.preprocessing import LabelEncoder
2 le_brand=LabelEncoder() #Create an object
3 #Understand the data and transform that to integer
4 cars['Brand']=le_brand.fit_transform(cars['Brand'])
```

```
In [51]: 1 from sklearn.preprocessing import LabelEncoder
2 le_model=LabelEncoder() #Create an object
3 #Understand the data and transform that to integer
4 cars['Model']=le_model.fit_transform(cars['Model'])
```

```
In [52]: 1 cars.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6019 entries, 0 to 6018
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Location               6019 non-null  int64
1   Year                  6019 non-null  int64
2   Kilometers_Driven     6019 non-null  int64
3   Fuel_Type             6019 non-null  int64
4   Transmission          6019 non-null  int64
5   Owner_Type            6019 non-null  int64
6   Mileage               6019 non-null  float64
7   Engine               6019 non-null  float64
8   Power                6019 non-null  float64
9   Seats                6019 non-null  float64
10  Price                6019 non-null  float64
11  Brand                6019 non-null  int64
12  Model                6019 non-null  int64
dtypes: float64(5), int64(8)
memory usage: 611.4 KB
```

```
In [53]: 1 cars
```

```
Out[53]:
```

	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine
0	9	2010	72000	0	1	0	26.60	998.0
1	10	2015	41000	1	1	0	19.67	1582.0
2	2	2011	46000	4	1	0	18.20	1199.0
3	2	2012	87000	1	1	0	20.77	1248.0
4	3	2013	40670	1	0	2	15.20	1968.0
...	...	...	...	...	...	...	...	...
6014	4	2014	27365	1	1	0	28.40	1248.0
6015	6	2015	100000	1	1	0	24.40	1120.0
6016	6	2012	55000	1	1	2	14.00	2498.0
6017	8	2013	46000	4	1	0	18.90	998.0
6018	5	2011	47000	1	1	0	25.44	936.0

6019 rows × 13 columns



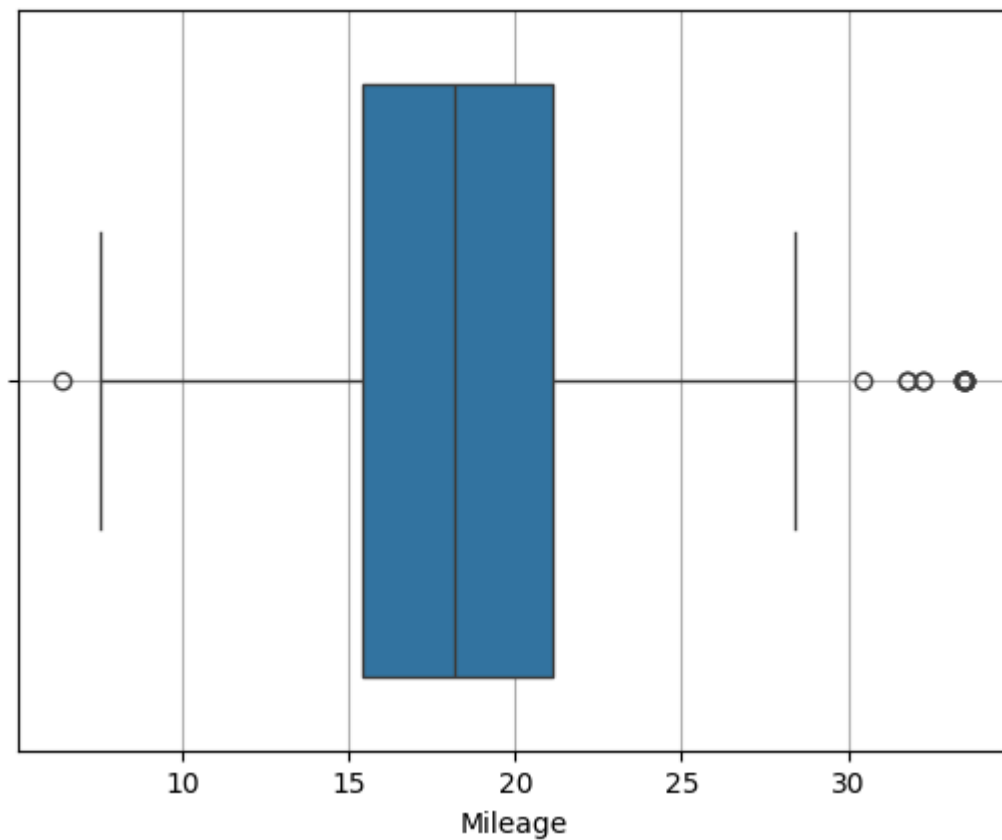
```
In [54]: 1 cars.to_csv('ML-cars-cleaned.csv',index=False)
```

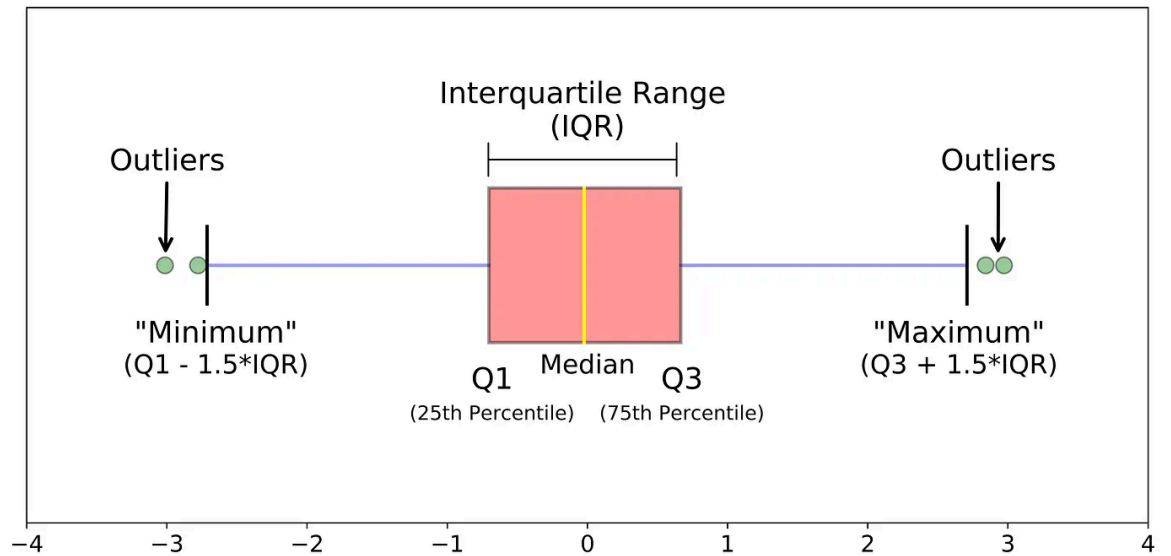

Find the outlier

1. An outlier is a data point that differs significantly from the rest of the observations in a dataset.
2. Because they deviate sharply from the general pattern, outliers can distort statistical analyses and machine learning models if not properly addressed
3. We can find the outlier using a boxplot

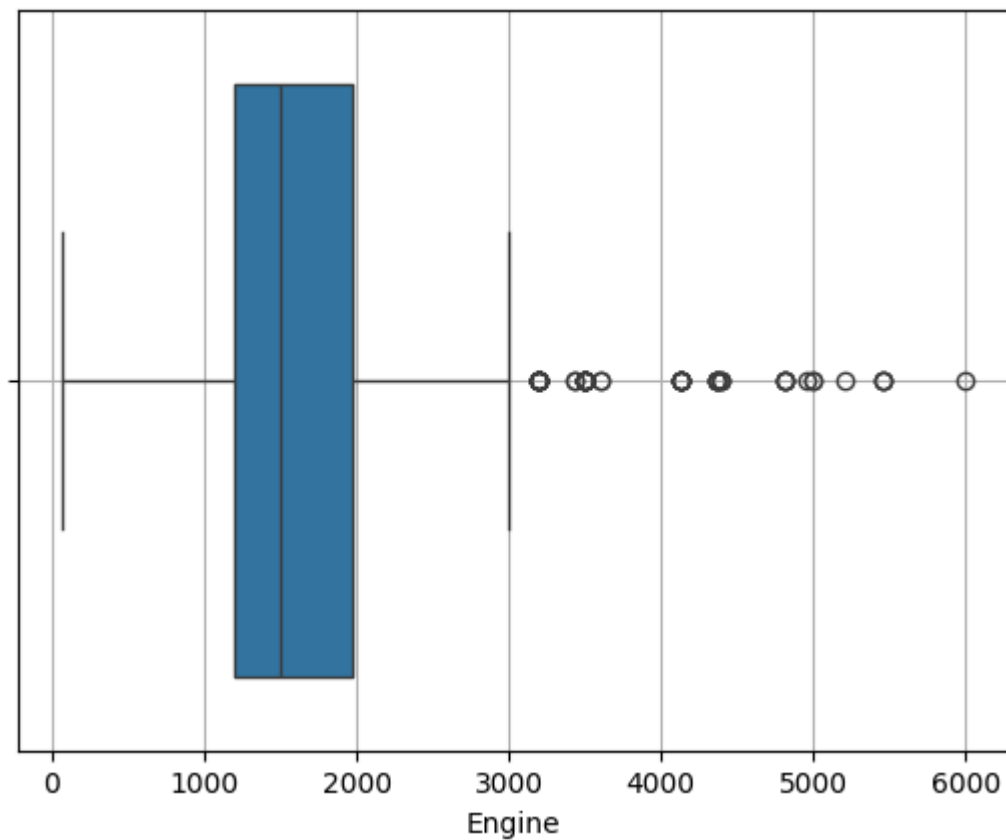
boxplot for Mileage

```
In [55]: 1 import seaborn as sns
          2 import matplotlib.pyplot as plt
          3 sns.boxplot(cars,x='Mileage')
          4 plt.grid()
          5 plt.show()
```





```
In [56]: 1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 sns.boxplot(cars,x='Engine')
4 plt.grid()
5 plt.show()
```



In [57]: 1 cars[cars['Engine']>5000]

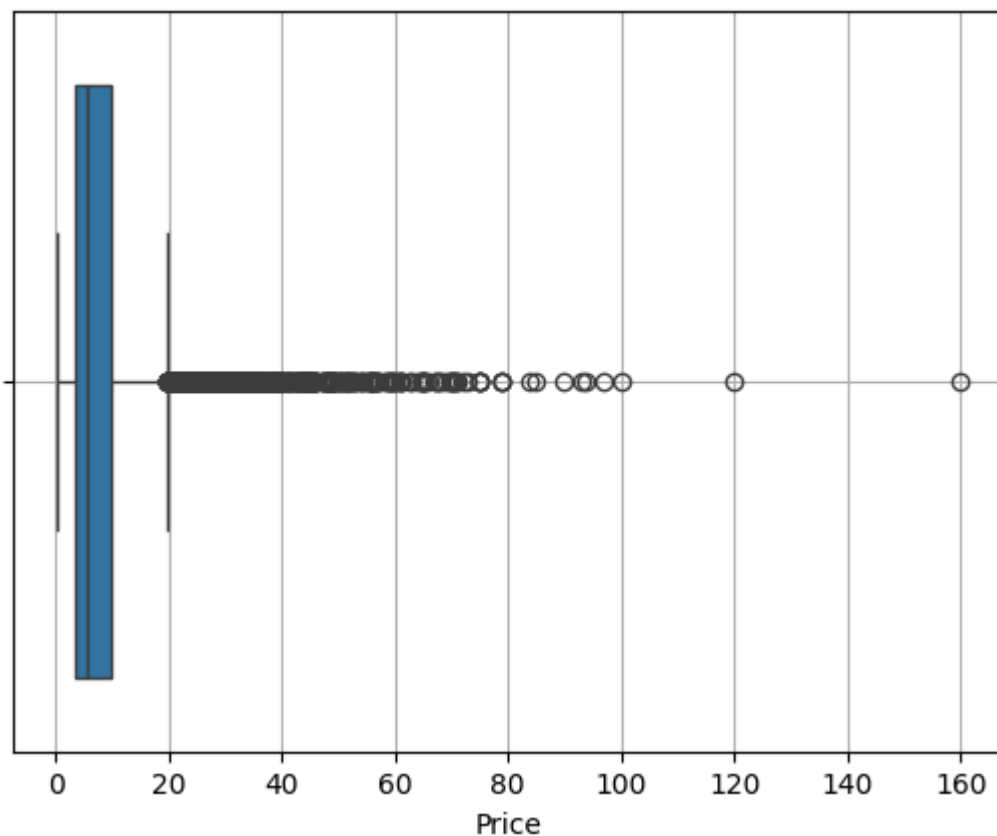
Out[57]:

	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine
152	8	2010	35277	4	0	0	7.81	5461.0
4691	1	2014	3000	4	0	2	12.00	5461.0
4722	8	2010	35000	4	0	0	8.10	5461.0
5521	5	2006	48000	4	0	0	8.60	5998.0
5781	4	2011	6500	4	0	3	6.40	5204.0

In [58]: 1 le_brand.classes_

Out[58]: array(['Ambassador', 'Audi', 'BMW', 'Bentley', 'Chevrolet', 'Datsun',
'Fiat', 'Force', 'Ford', 'Honda', 'Hyundai', 'ISUZU', 'Isuzu',
'Jaguar', 'Jeep', 'Lamborghini', 'Land', 'Mahindra', 'Maruti',
'Mercedes-Benz', 'Mini', 'Mitsubishi', 'Nissan', 'Porsche',
'Renault', 'Skoda', 'Smart', 'Tata', 'Toyota', 'Volkswagen',
'Volvo'], dtype=object)

In [59]: 1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 sns.boxplot(cars,x='Price')
4 plt.grid()
5 plt.show()



Correlation coefficient - is a statistical metric that measures the strength and direction of a linear relationship between two

. . .

In [60]: 1 cars.corr()

Out[60]:

	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Typ
Location	1.000000	-0.020117	-0.042751	0.115048	0.040174	-0.03388
Year	-0.020117	1.000000	-0.173048	-0.129526	-0.097059	-0.38616
Kilometers_Driven	-0.042751	-0.173048	1.000000	-0.102953	0.025663	0.07992
Fuel_Type	0.115048	-0.129526	-0.102953	1.000000	0.125219	0.04301
Transmission	0.040174	-0.097059	0.025663	0.125219	1.000000	0.00300
Owner_Type	-0.033889	-0.386160	0.079924	0.043017	0.003002	1.00000
Mileage	0.010125	0.291291	-0.064275	-0.153273	0.358682	-0.14605
Engine	-0.078302	-0.051712	0.091029	-0.401013	-0.499082	0.04852
Power	-0.065012	0.015240	0.033248	-0.264078	-0.639173	0.02808
Seats	-0.037596	0.011795	0.084163	-0.297830	0.073794	0.00320
Price	-0.116620	0.305327	-0.011493	-0.305207	-0.586025	-0.09455
Brand	0.026295	0.018578	0.028232	-0.068246	0.168697	-0.02758
Model	0.005194	-0.027290	0.034198	-0.033179	0.174854	-0.00032

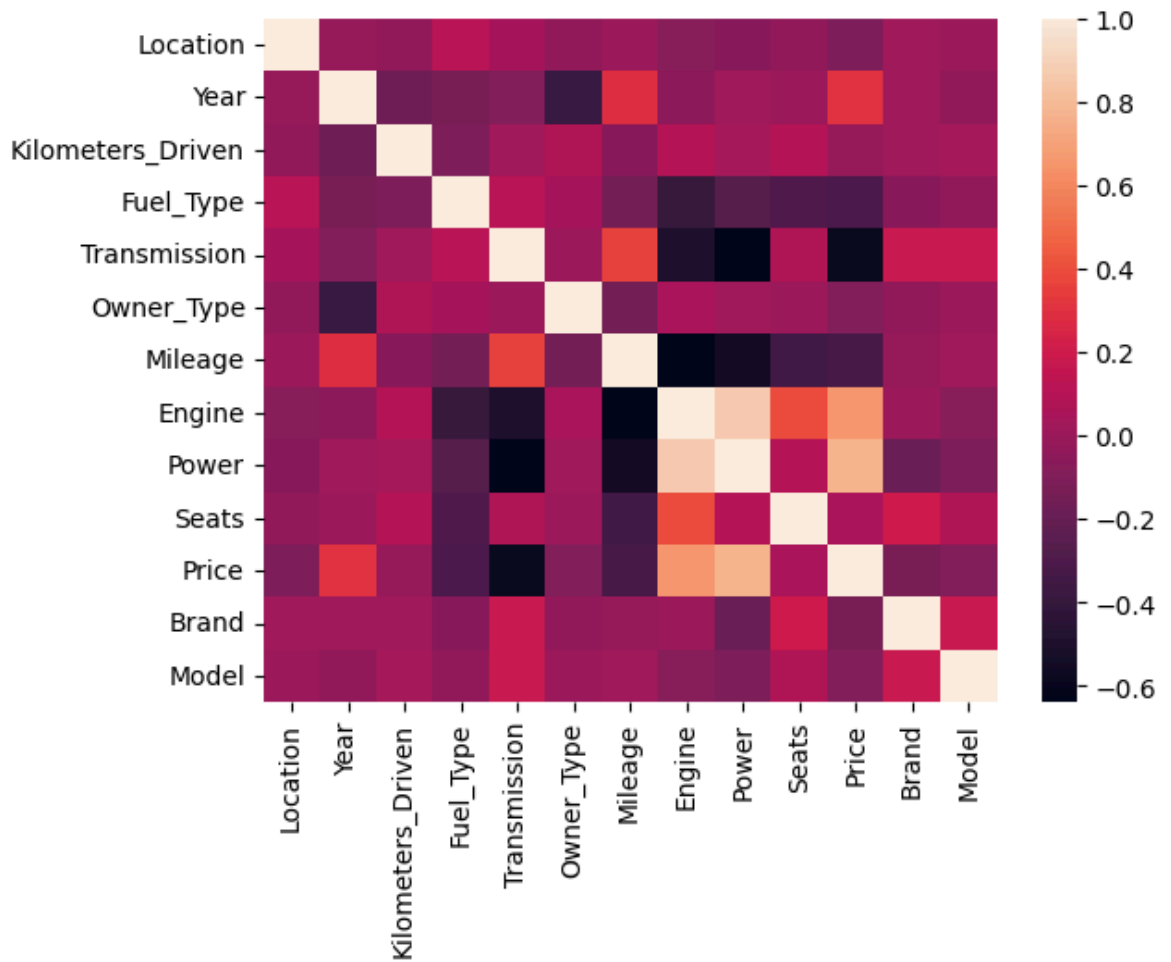


In [61]: 1 cars.corr()['Price']

```
Out[61]: Location      -0.116620
Year                0.305327
Kilometers_Driven  -0.011493
Fuel_Type          -0.305207
Transmission       -0.586025
Owner_Type         -0.094559
Mileage            -0.329988
Engine              0.657118
Power              0.767727
Seats              0.053136
Price              1.000000
Brand              -0.132929
Model              -0.088536
Name: Price, dtype: float64
```

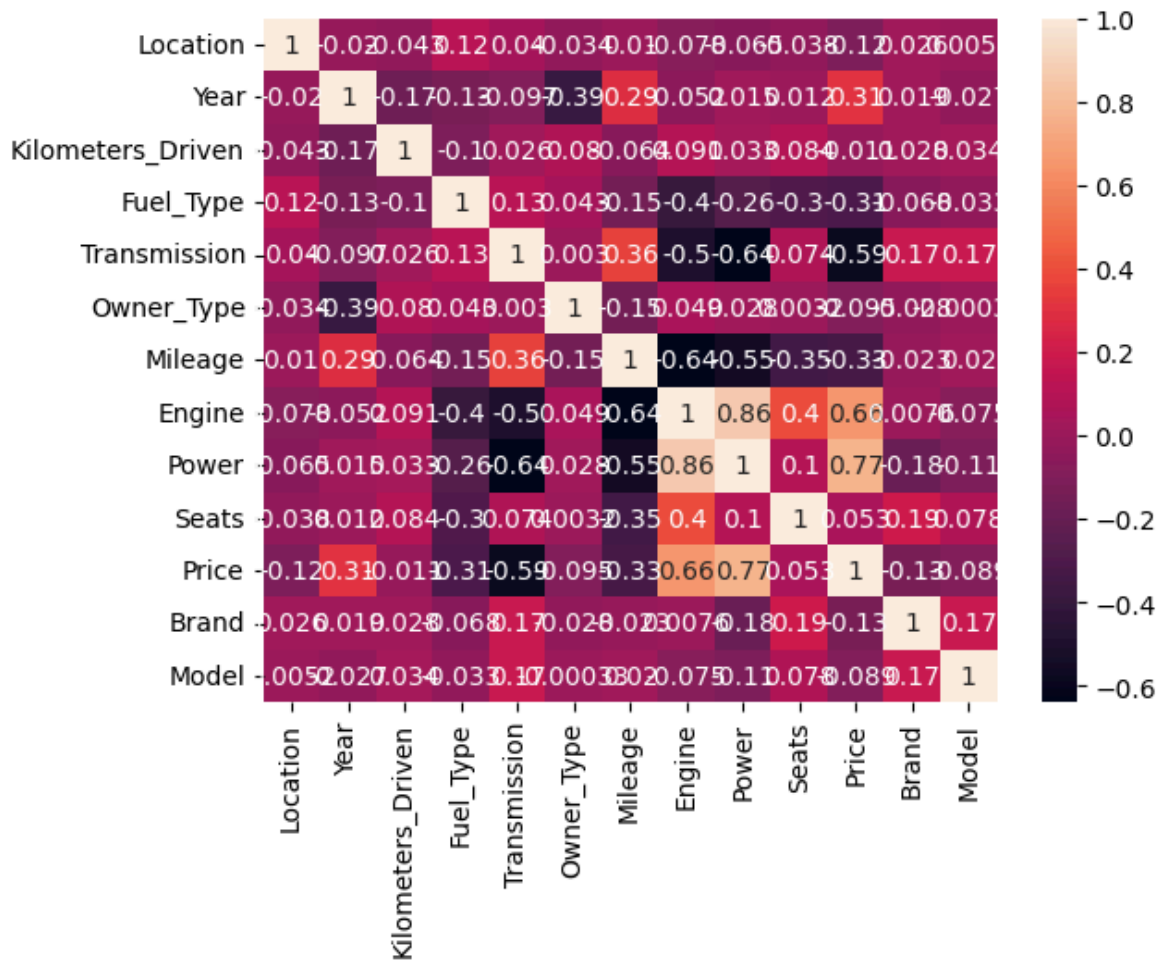
```
In [62]: 1 import seaborn as sns  
        2 sns.heatmap(cars.corr())
```

Out[62]: <Axes: >

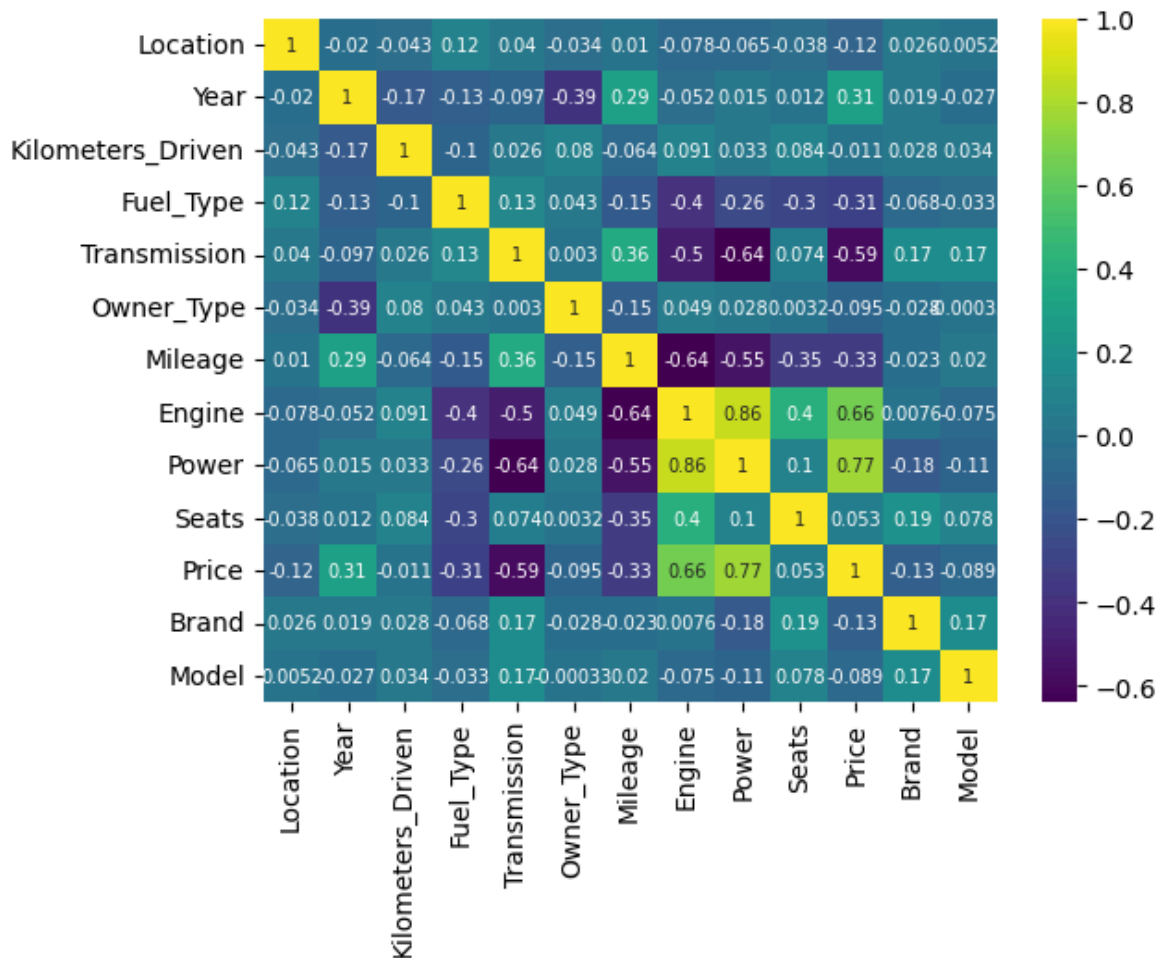


```
In [65]: 1 import seaborn as sns
          2 sns.heatmap(cars.corr(),annot=True)
```

Out[65]: <Axes: >



```
In [73]: 1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 sns.heatmap(cars.corr(),annot=True, cmap="viridis",annot_kws={"size": 7})
4 plt.figure(figsize=(70,70))
5 plt.show()
```

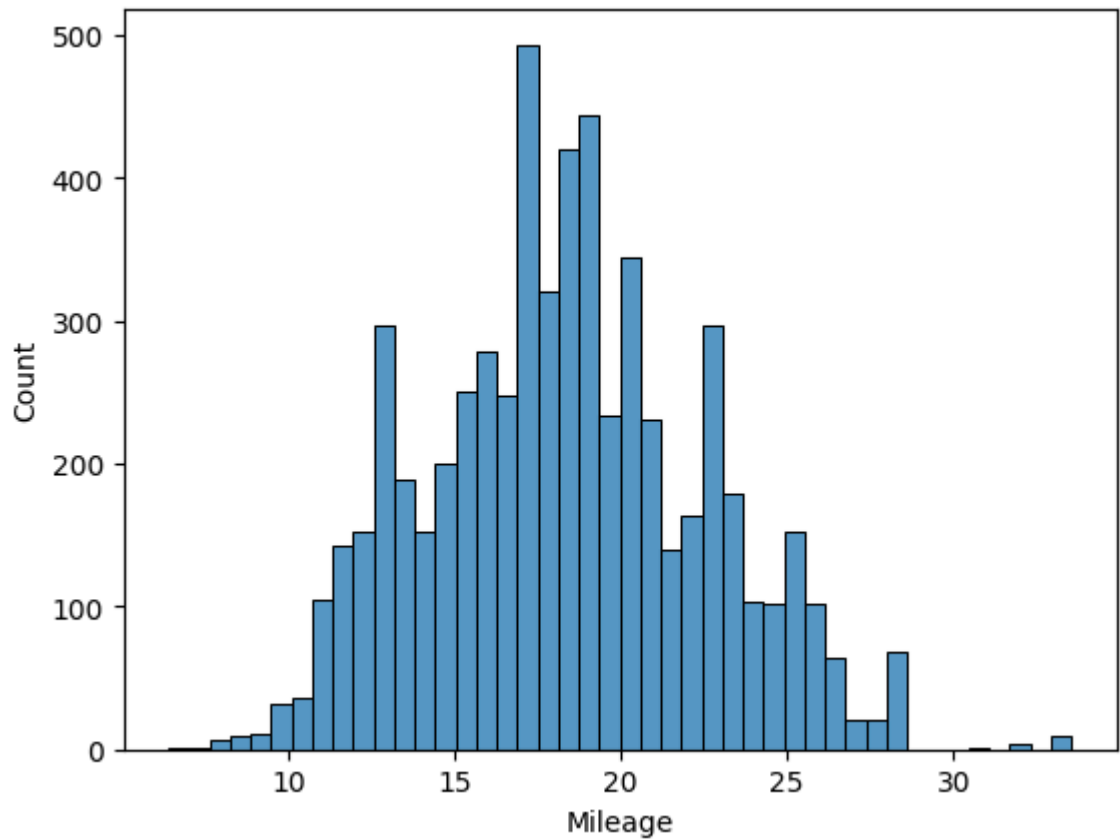


<Figure size 7000x7000 with 0 Axes>

Distribution Analysis

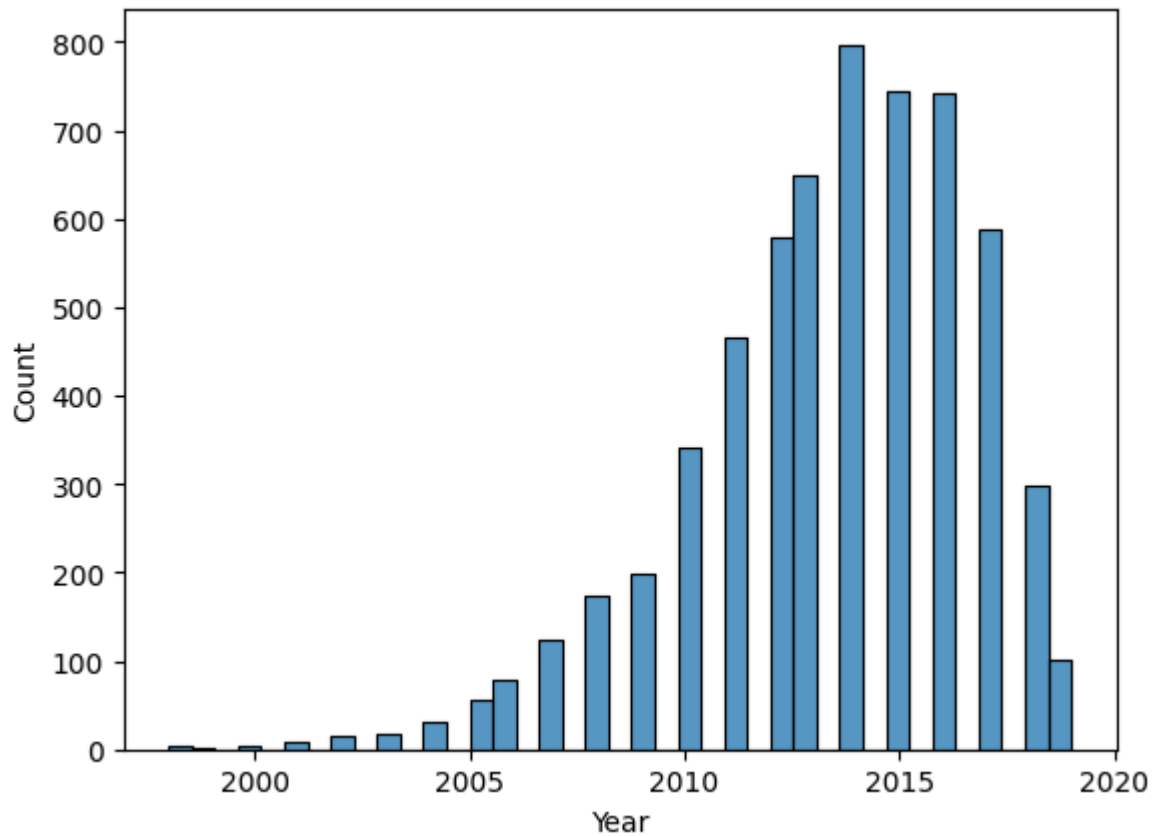
```
In [74]: 1 import seaborn as sns  
        2 sns.histplot(cars['Mileage'])
```

Out[74]: <Axes: xlabel='Mileage', ylabel='Count'>




```
In [75]: 1 import seaborn as sns  
        2 sns.histplot(cars['Year'])
```

Out[75]: <Axes: xlabel='Year', ylabel='Count'>



```
In [ ]: 1
```